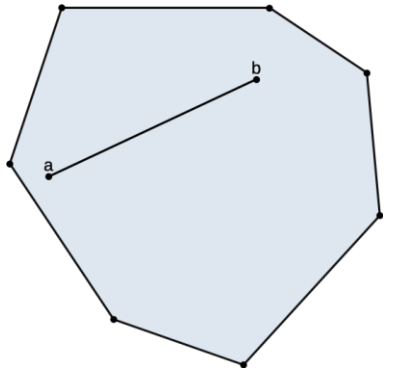
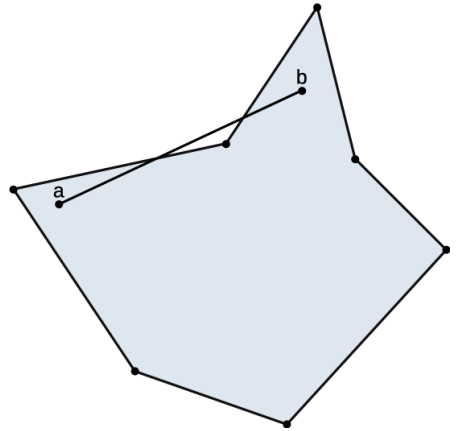
Competitive Programming

Basic Geometry

Convex Polygons

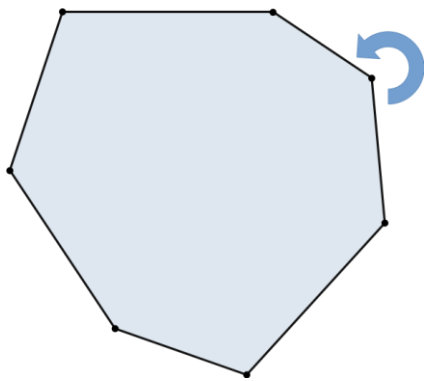
- A convex polygon looks like this.
  - If you choose any two points inside the polygon, all points between them are also inside.
  - Not all polygons are like this.
- Convex polygons are nice to work with.
- Let’s say we have a list ordered list of points defining the perimeter.
  - Can we tell if it’s a convex polygon?

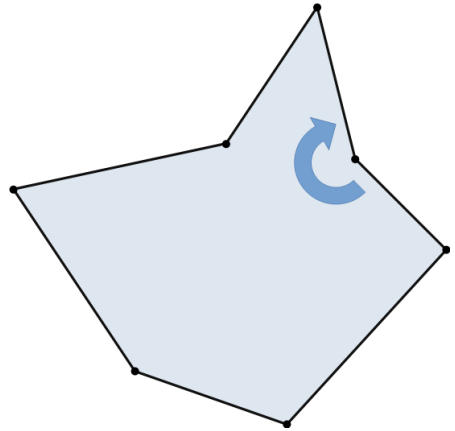




Detecting Convex Polygons

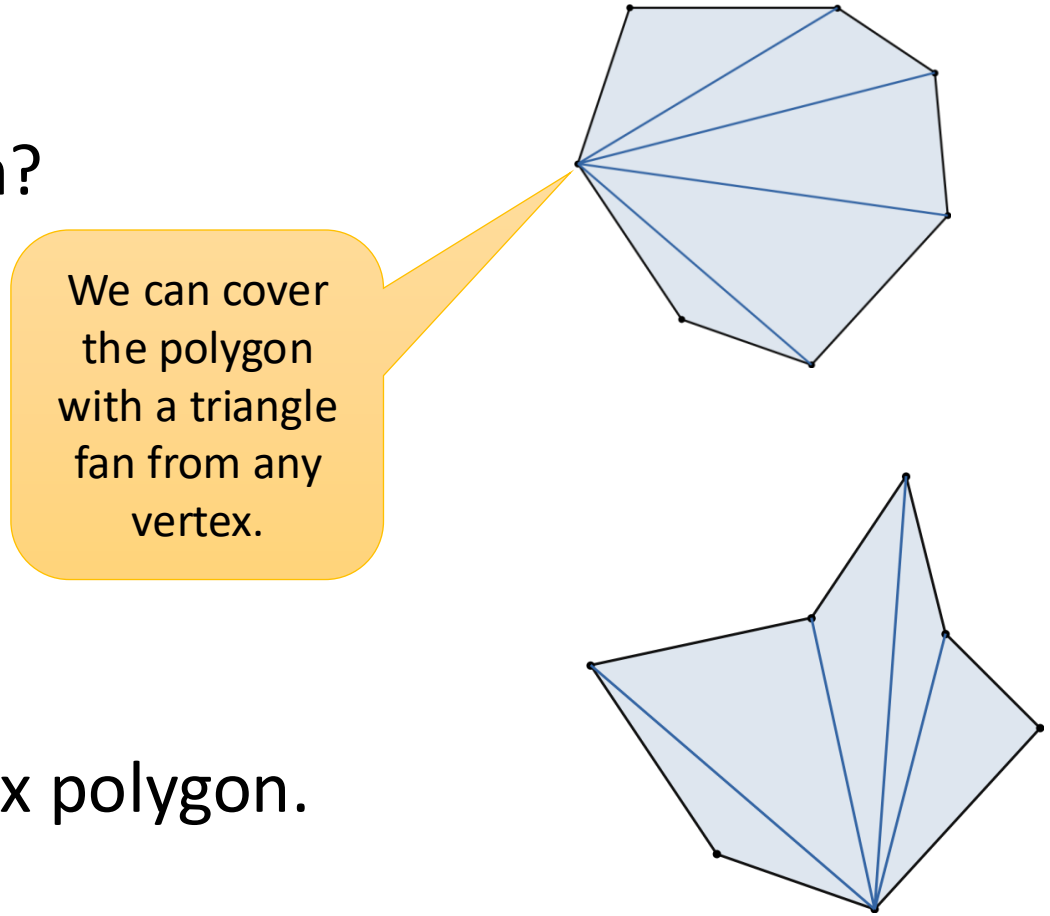
- Let's say we have a counter-clockwise ordered list of n points defining its perimeter.
- If we walk around the perimeter, we never have to make a right turn.
- That's a good job for cross product … a constant-time operation.
- That's a O(n) algorithm.





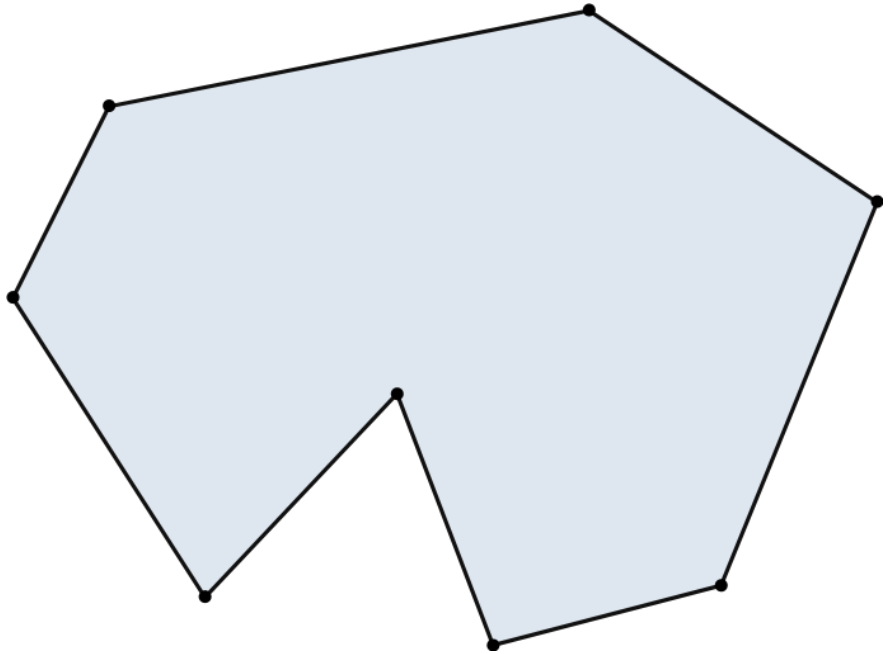
Area of a Convex Polygon

- What if we want the area of a polygon?
- Convex polygons are easy to tessellate into triangles.
  - Then, we can compute their area as the sum of the triangle areas.
  - Each of which is easy to compute with cross product.
  - A O(n) time algorithm.
- Still possible to tessellate a non-convex polygon.
  - Then add up the areas of the triangles.
  - … but this is going to be tricky.



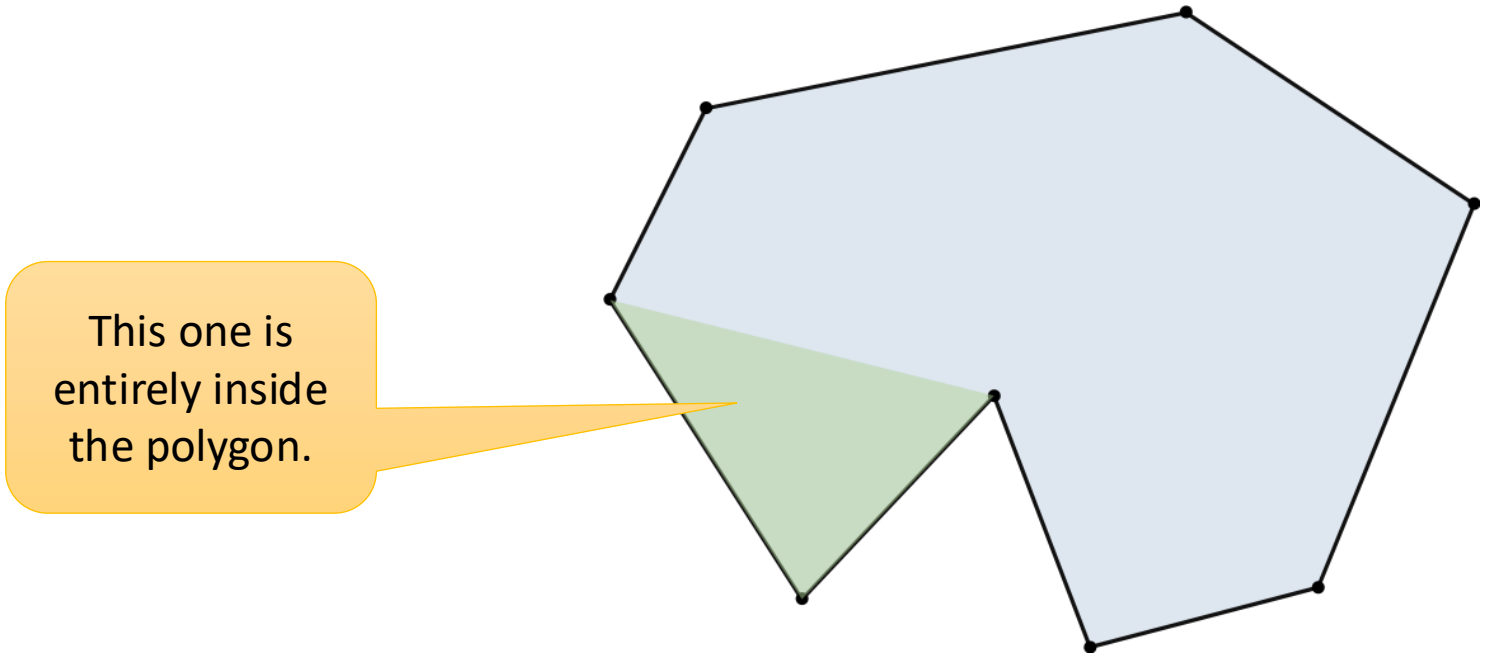
Area of a Simple Polygon

- We can get the area of a simple, non-convex polygon the same way we did for a convex one.
- This is called the surveyor's algorithm.



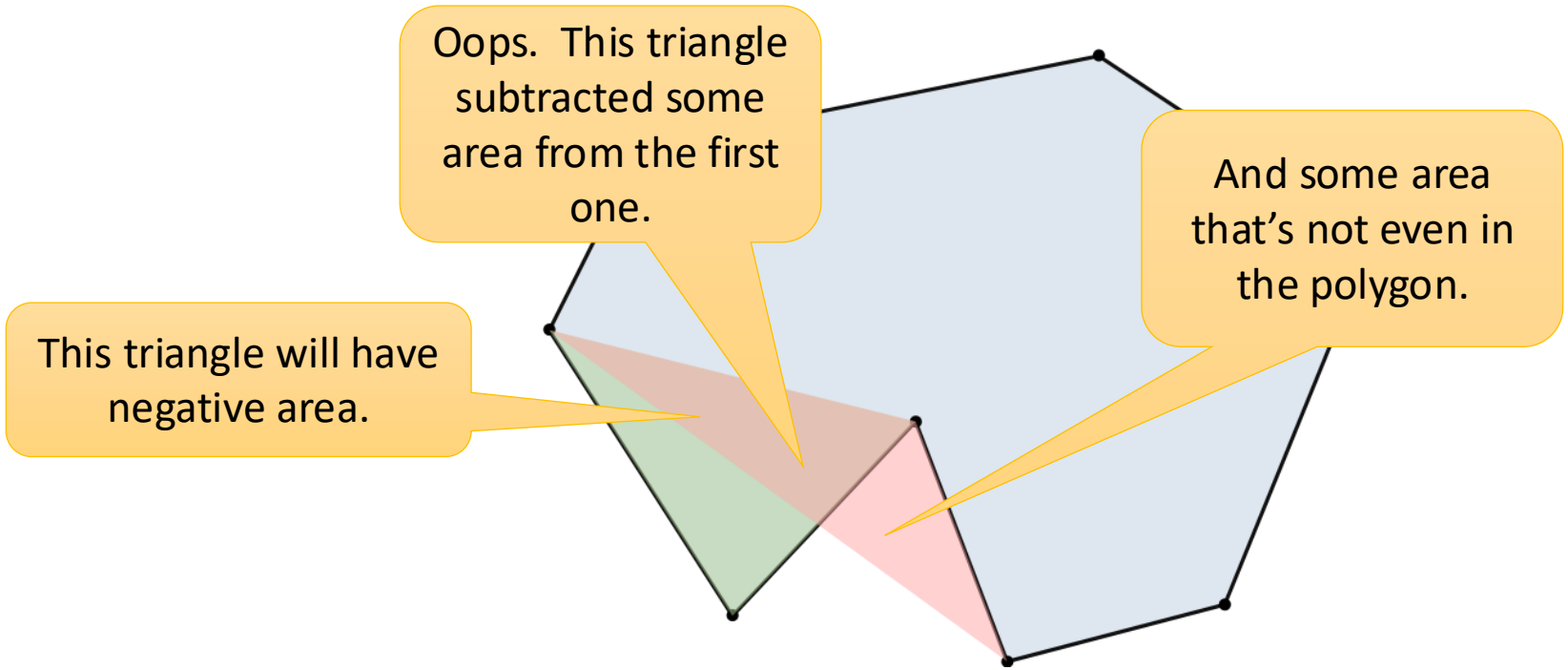
Surveyor’s Algorithm

- Just add up all the (signed) areas for a triangle fan from any vertex.



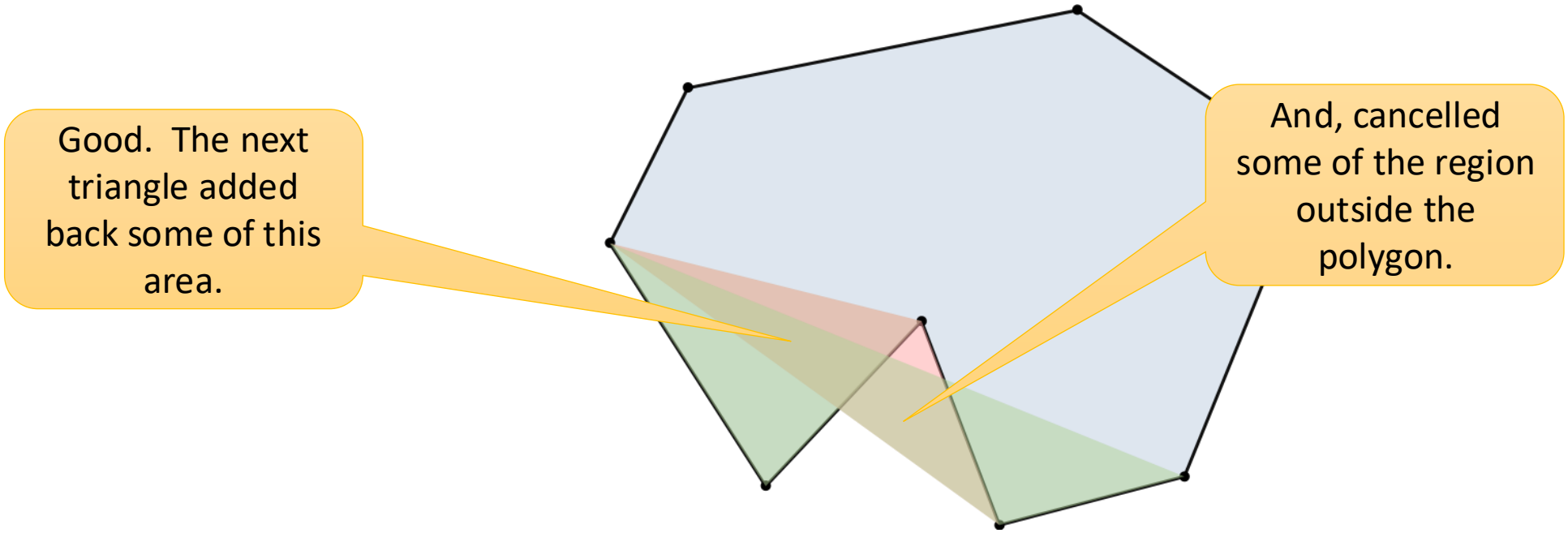
Surveyor's Algorithm

- Just add up all the (signed) areas for a triangle fan from any vertex.



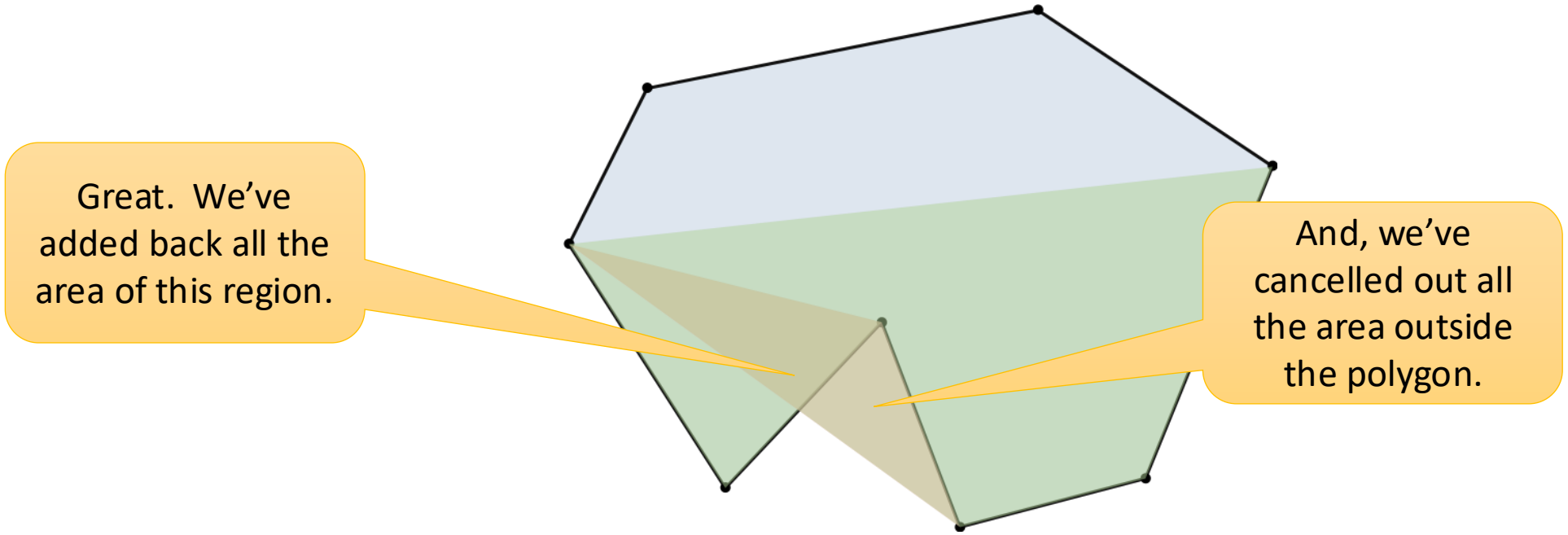
Surveyor's Algorithm

- Just add up all the (signed) areas for a triangle fan from any vertex.



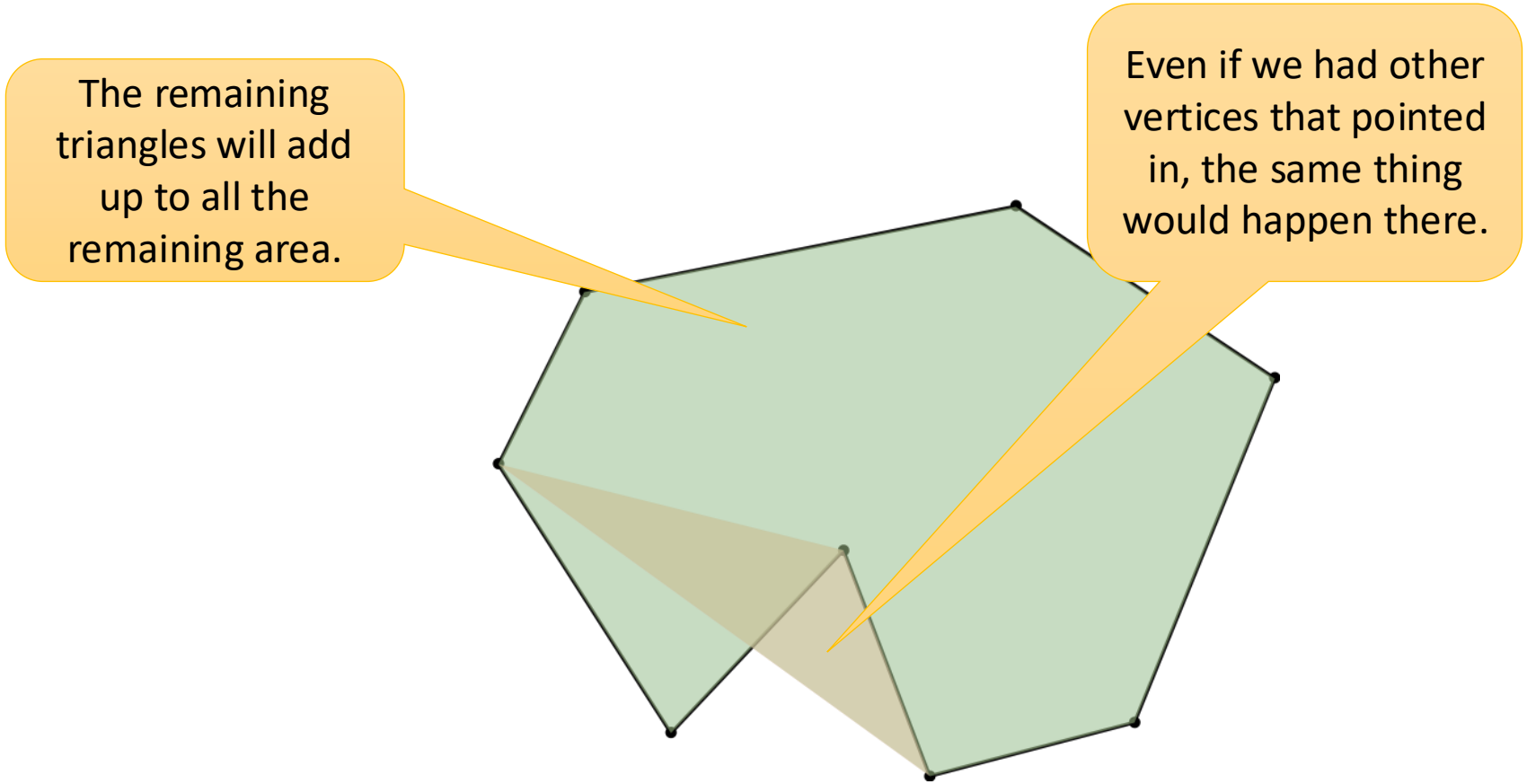
Surveyor's Algorithm

- Just add up all the (signed) areas for a triangle fan from any vertex.



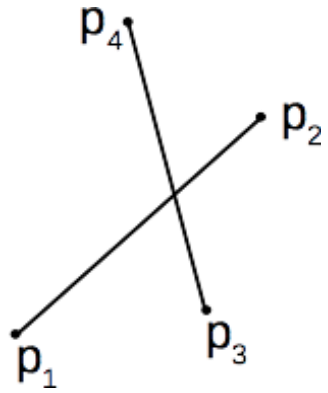
Surveyor's Algorithm

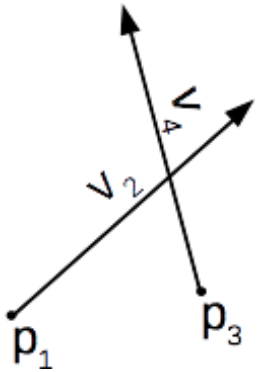
- Just add up all the (signed) areas for a triangle fan from any vertex.



Dot Product for line segment intersection

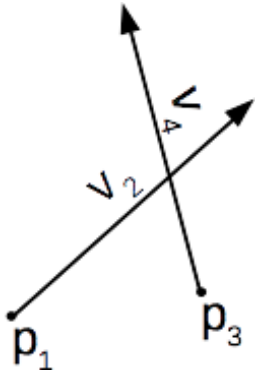
- Consider two line segments, $\overline{p_1 p_2}$ and $\overline{p_3 p_4}$
- Do they intersect? Where?
- This is a good application of:
  - A parametric representation
  - Dot product
- We use the dot product to find an intersection point for two lines.
- The value of t_4 where $p_3 + t_4 v_4$ crosses the other line segment.
- Let's give it a try.

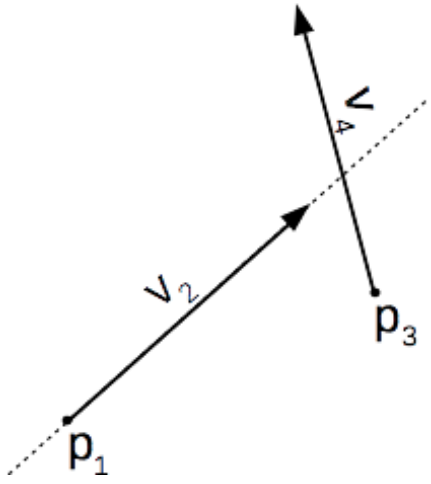




Dot Product for line segment intersection

- We use the dot product to find an intersection point.
- The value of t_4 where $p_3 + t_4 v_4$ crosses the other line segment.
- It's $$t_4 = \frac{(p_1 - p_3) \cdot v_2^{\perp}}{v_4 \cdot v_2^{\perp}}$$
- But we also have to consider this case:



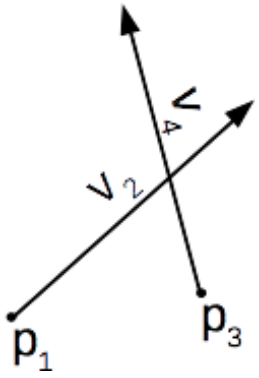


Dot Product for line segment intersection

- We'll need to compute both of these:

$$t_4 = \frac{(p_1 - p_3) \cdot v_2^{\perp}}{v_4 \cdot v_2^{\perp}}$$

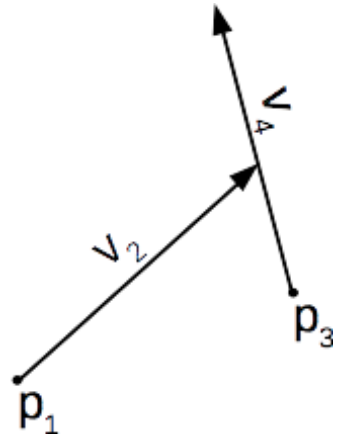
$$t_2 = \frac{(p_3 - p_1) \cdot v_4^{\perp}}{v_2 \cdot v_4^{\perp}}$$



- And make sure they're both between 0 and 1.

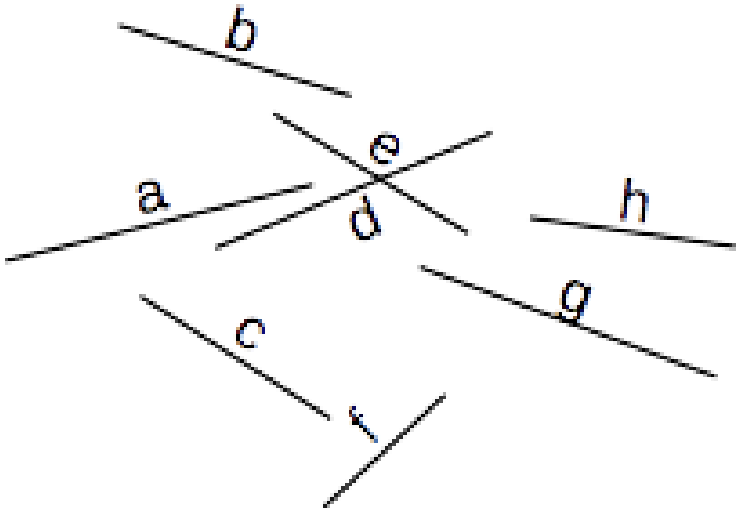
Dot Product for line segment intersection

- Could be very sensitive in cases like this:
- With the division operator, rounding error could make it hard to detect when segments cross at (or very near) an endpoint.
- There are a couple of ways to fix this:
  - We could add a little margin for error
  - Multiply through to avoid division



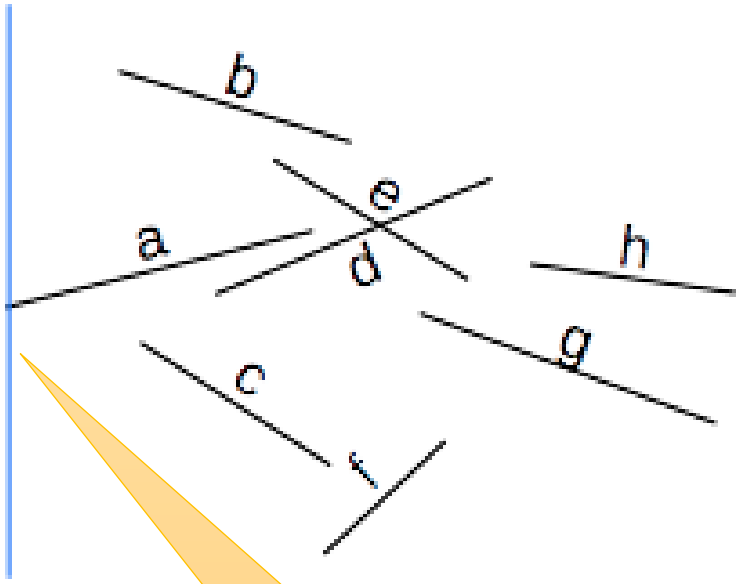
Multiple Lines

- What if we have a set of n lines
  - And we want to see if any of them intersect.
- Easy, just run intersection detection on all pairs.
  - That's $\Theta(n^2)$
  - Done
    …. not really.



Multiple Lines via Line Sweep

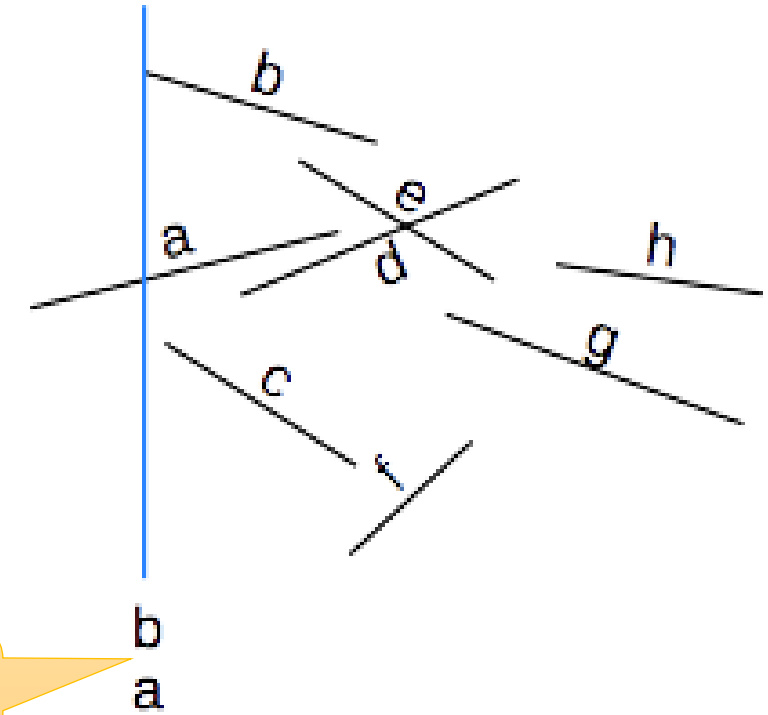
- We can do this faster using a *line sweep* technique.
  - Start with a sweep line at the left edge of the figure.
  - Work your way to the right stopping to consider events of interest.
  - Here, events are the start and end of line segments.



Nothing interesting will happen until we reach the left edge of b.

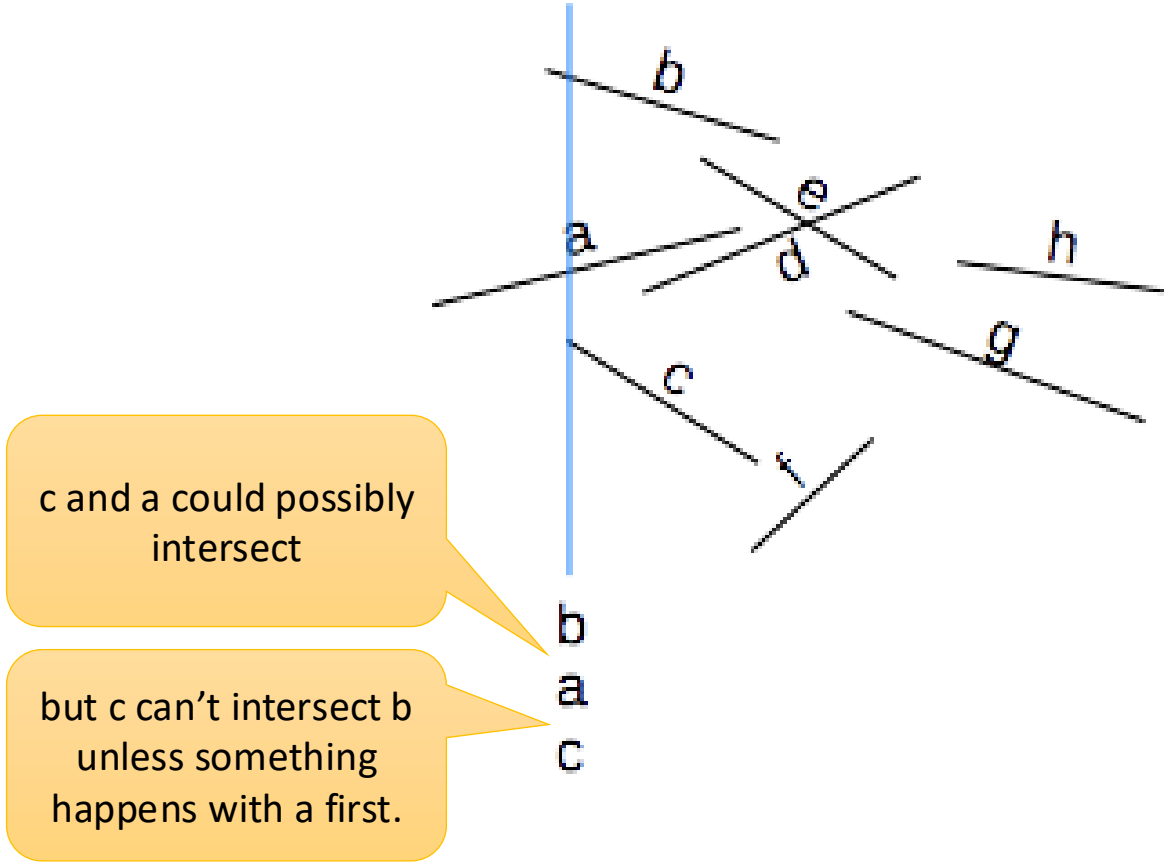
Multiple Lines via Line Sweep

- We can order the segments vertically
  - By where they hit the sweep line.
- Unless lines cross, this order will only change when we hit an event of interest.
  - The start or end of a line segment.



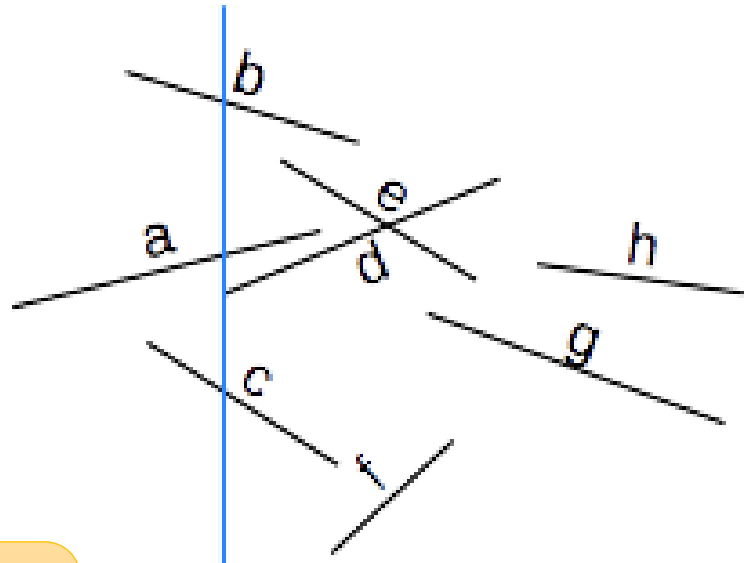
Multiple Lines via Line Sweep

- We only need to check for intersections with neighbors in the sweep line order.



Multiple Lines via Line Sweep

- Continuing …

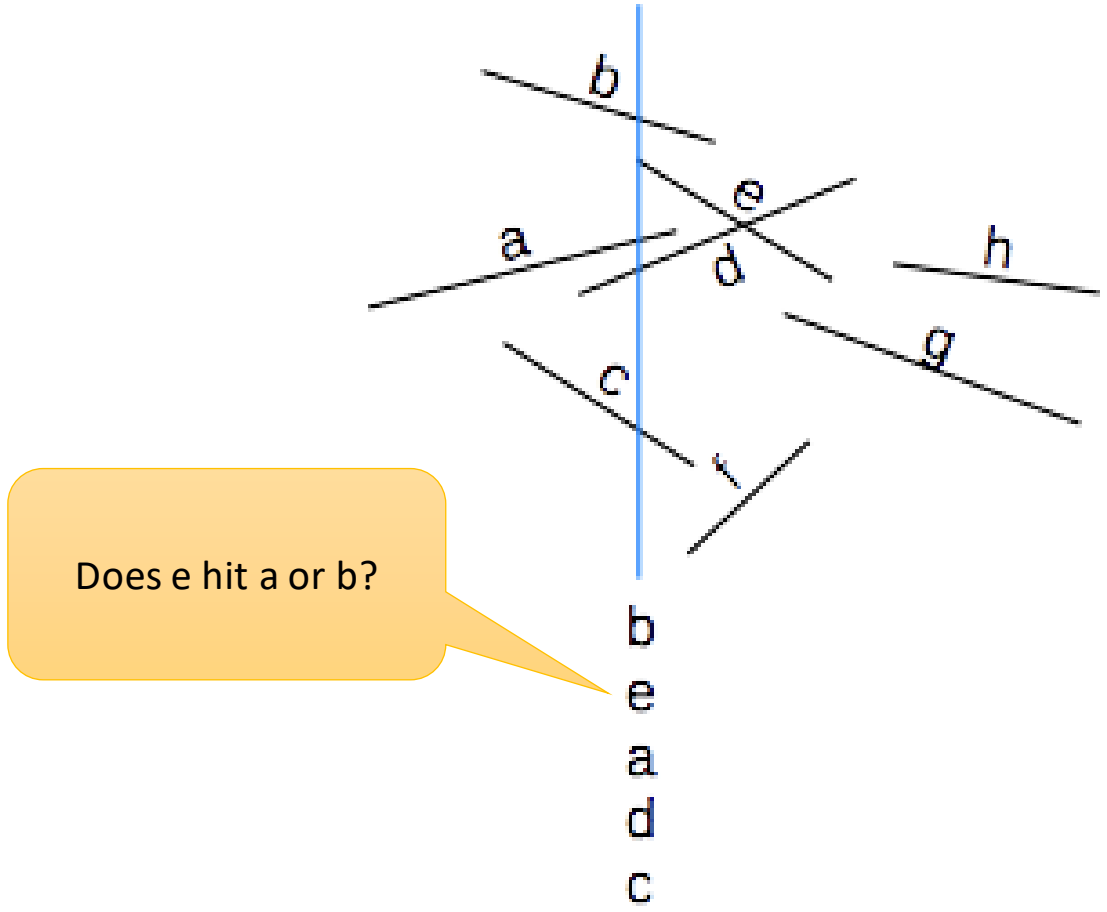


Does d hit c or a?
No?

b
a
d
c

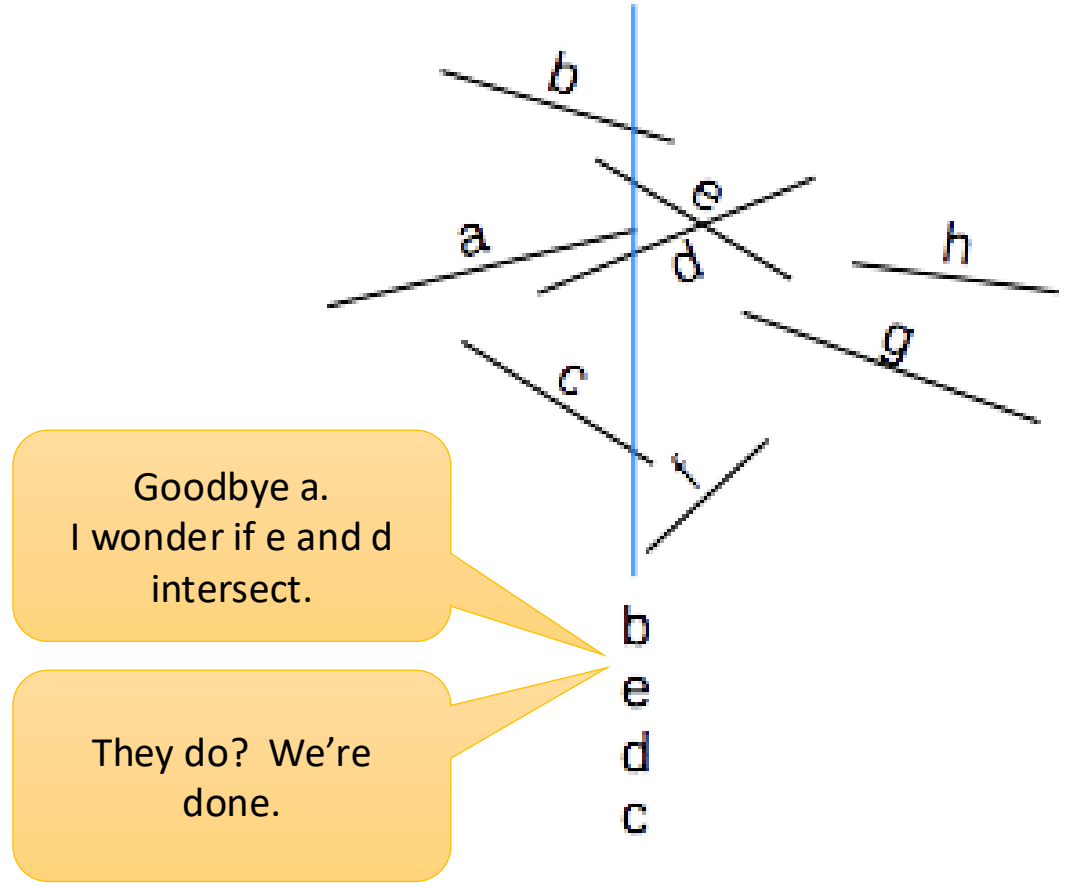
Multiple Lines via Line Sweep

- Continuing …



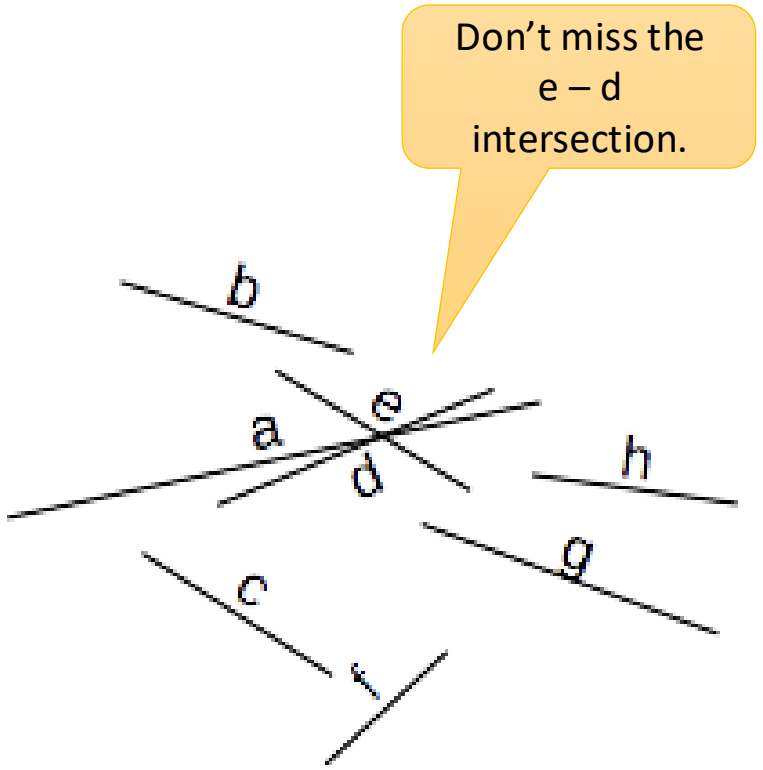
Multiple Lines via Line Sweep

- When segments leave the order, that might might make new neighbors.
  - We have to check them.



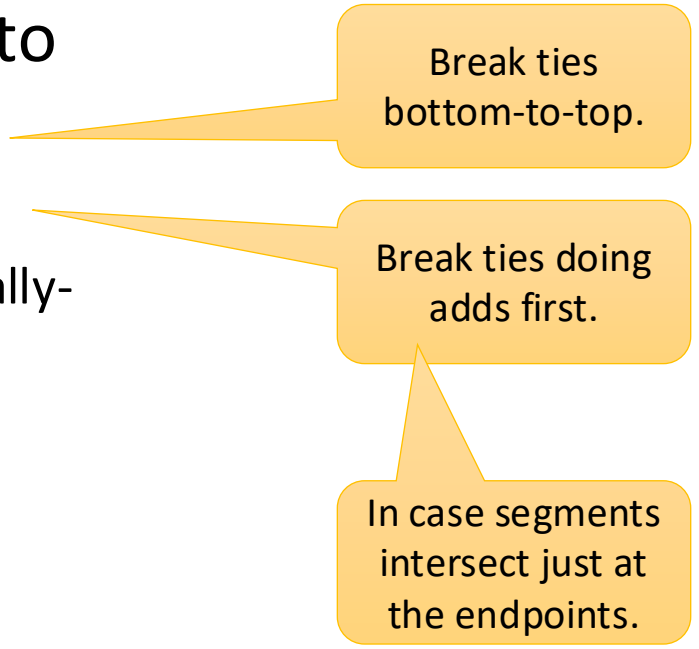
Enhancing Line Sweep

- This technique detects one intersection
  - But we could extend it to find all of them.
  - … but, that would affect the runtime.
  - there could be up to $\Omega(n^2)$ intersections.
- Observe, this technique would have trouble with vertical lines
  - But, we could extend it to handle them
- And, it would have a little trouble if three segments intersected at the same point.
  - But, we could fix that also.



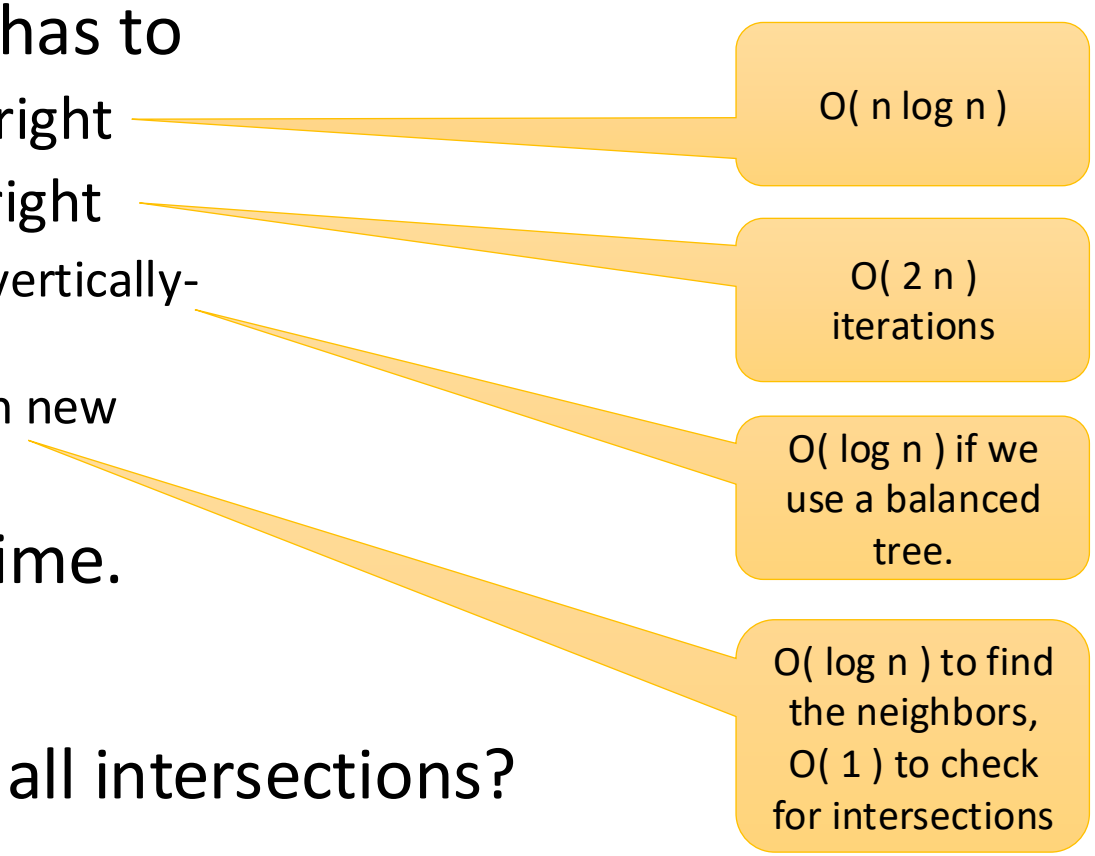
Analysis

- The line sweep algorithm has to
  - Sort the endpoints left-to-right
  - Process endpoints left-to-right
    - Insert or remove from our vertically-sorted list of segments
    - Check for intersections with new neighbors.



Analysis

- The line sweep algorithm has to
  - Sort the endpoints left-to-right
  - Process endpoints left-to-right
    - Insert or remove from our vertically-sorted list of segments
    - Check for intersections with new neighbors.
- So, that's O(n log n) runtime.
- What if you had to report all intersections?



Line Sweep / Plane Seep Techniques

- Line sweep can be applied to lots of problems in two (or more) dimensions (plane sweep)
- A problem may look tricky in general, but be easy in little slices.
- For example, area of a non-convex polygon.
  - Vertices are the events of interest.
  - We can add up all the areas between the current and previous sweep positions.

